



Production Systems Owner's Handbook

A complete guide to Langmuir's in-house production management platform: what each system does, how they connect, how changes are made safely, and how to run it all with Claude Cowork as your engineering partner.

BEFORE YOU START

How to Use This Handbook

You are inheriting a family of small, purpose-built web applications that run Langmuir's production floor: production boards, warehouse picking, receiving, shipping verification, cycle counting, tooling purchases, and a scheduling engine that is growing into a lean MRP. None of it is off-the-shelf software. All of it was built in-house, deliberately simple, specifically so that a non-developer can own it.

You do not need to be a programmer to run this system. You need three things, and this handbook is designed to give you all of them:

1. **A mental map** — what each application does, who on the floor uses it, and where its data lives (Parts 1–5).
2. **A working method** — how to make changes safely using Claude Cowork, where you describe what you want in plain English, review what Claude proposes, and approve it (Part 6).
3. **An operational routine** — what runs automatically every day, how to check that everything is healthy, and what to do when something looks wrong (Part 7).

Your learning setup

You have four resources to learn against, and they reinforce each other. First, **this handbook** — read it front to back once, then use it as a reference. Second, **the Cowork workspace folder** — the exact same folder Brendan uses, containing every application's source code and design documents; you can open any file mentioned here and read it alongside the explanation. Third, **the live sites** — every application is running right now; open the pages as you read about them and click around. Fourth, **the GitHub repositories and the Railway project** — where code is stored and where the applications actually run.

A suggested path through the material

Read Part 1 first — it is the map everything else hangs on. Then read Part 6 (working with Cowork) even though it comes later, because the fastest way to learn this system is to ask Claude questions about it inside the workspace: "explain what happens when a picker fulfills a request" is a perfectly good prompt, and Claude will answer from the actual code. Then work through the application chapters in Part 3 with the live sites open. Parts 4 and 5 (Apps Scripts and Epicor) are reference-heavy; skim them once and return when needed. Part 7 is your day-to-day checklist. Part 8 tells you what was in flight at handover so nothing catches you by surprise.

THE ONE RULE THAT MATTERS MOST

Everything in this workspace is the **real, live production system** — real customer data, real inventory, real floor screens. There is a separate, anonymized demo world ("Finn Operations") that lives in a different folder and is *not* part of this handover's day-to-day. Never copy real customer names, real work-order data, or Langmuir branding into anything demo-related, and when in doubt about which world a request touches, stop and check. Section 6.5 covers this in detail.

Contents

Part 1 — The Big Picture	5
1.1 What you're inheriting	5
1.2 The system map	6
1.3 The design philosophy	6
1.4 The three data stores	7
Part 2 — Infrastructure & How Changes Go Live	9
2.1 Where everything lives	9
2.2 The registry: config.sh	9
2.3 The life of a code change	10
2.4 Apps Script deployments (and the versioning gotcha)	11
2.5 Helper scripts reference	11
2.6 Access & security model	12
Part 3 — The Applications	13
3.1 PMS — the hub of the floor	13
3.2 KPI — the wall board	15
3.3 Cycle Count — and its migration into PMS	15
3.4 Tooling — AI-assisted purchase requests	16
3.5 Vision — part identification by photo	16
3.6 Scheduling / MRP — the planning brain	17
3.7 Hub & Procurement	18
3.8 PMS-Test — the staging line	18
Part 4 — The Google Apps Scripts	19
Part 5 — The Epicor Connection (BAQs)	21
Part 6 — Working with Claude Cowork	23
6.1 What the workspace is	23
6.2 The working loop	23
6.3 Understanding impact before you approve	24
6.4 How to ask well	25
6.5 Two worlds: real Langmuir vs. the showroom	25
6.6 Hard rules	25
Part 7 — Operational Playbook	27
7.1 What runs automatically	27
7.2 Health checks & diagnostics	27
7.3 Common failure modes and fixes	28

7.4 Your first two weeks	28
Part 8 — In-Flight Projects at Handover	30
Appendix A — Service & URL reference	32
Appendix B — Environment variable reference	33
Appendix C — Glossary	34

The Big Picture

1.1 What you're inheriting

Langmuir's production floor runs on a family of roughly nine small web applications, each one a plain Node.js/Express server hosted on **Railway** (a cloud platform that runs the apps and restarts them automatically when code changes). Each app has one clear job:

App	One-line job	Main users
PMS	The floor hub: production boards, part requests, warehouse picking, receiving, shipping verification, cycle counting, engineering diagrams, feedback	Everyone on the floor
KPI	Wall-display dashboard: finished goods by line, shipping stats, continuous-improvement board	Whole floor (TV), managers
Cycle Count	Daily physical-count portal (being absorbed into PMS — see 3.3)	Counters, ops director
Tooling	Request board for off-the-shelf tools/supplies, with AI-suggested vendor options	Operators, purchaser
Vision	Behind-the-scenes photo-based part identification for Receiving	(No direct users — serves PMS)
Scheduling / MRP	The planning brain: daily build plans per line, demand forecasting, material projections, recommended POs, staffing plans	Line leads, production manager
Hub	A simple landing page linking to every internal tool	Everyone
Procurement	Reorder-point and stockout-risk decision support for purchasing	Purchasing
PMS-Test	Staging environment — a v2 rewrite of the PMS core, for safe experimentation	You and Claude

Around these apps sit three supporting layers. **Epicor Kinetic** is the company ERP and the ultimate source of truth for parts, inventory, jobs, and purchase orders; the apps read from it (never write to it) through saved queries called **BAQs**. **Google Sheets with Apps Scripts** act as both a live inventory ledger the floor manager can open and edit, and as data bridges that pull Epicor data on schedules. **Railway Postgres databases** store application state — pick logs, shipping carts, count sessions, PDF diagrams, plans.

1.2 The system map

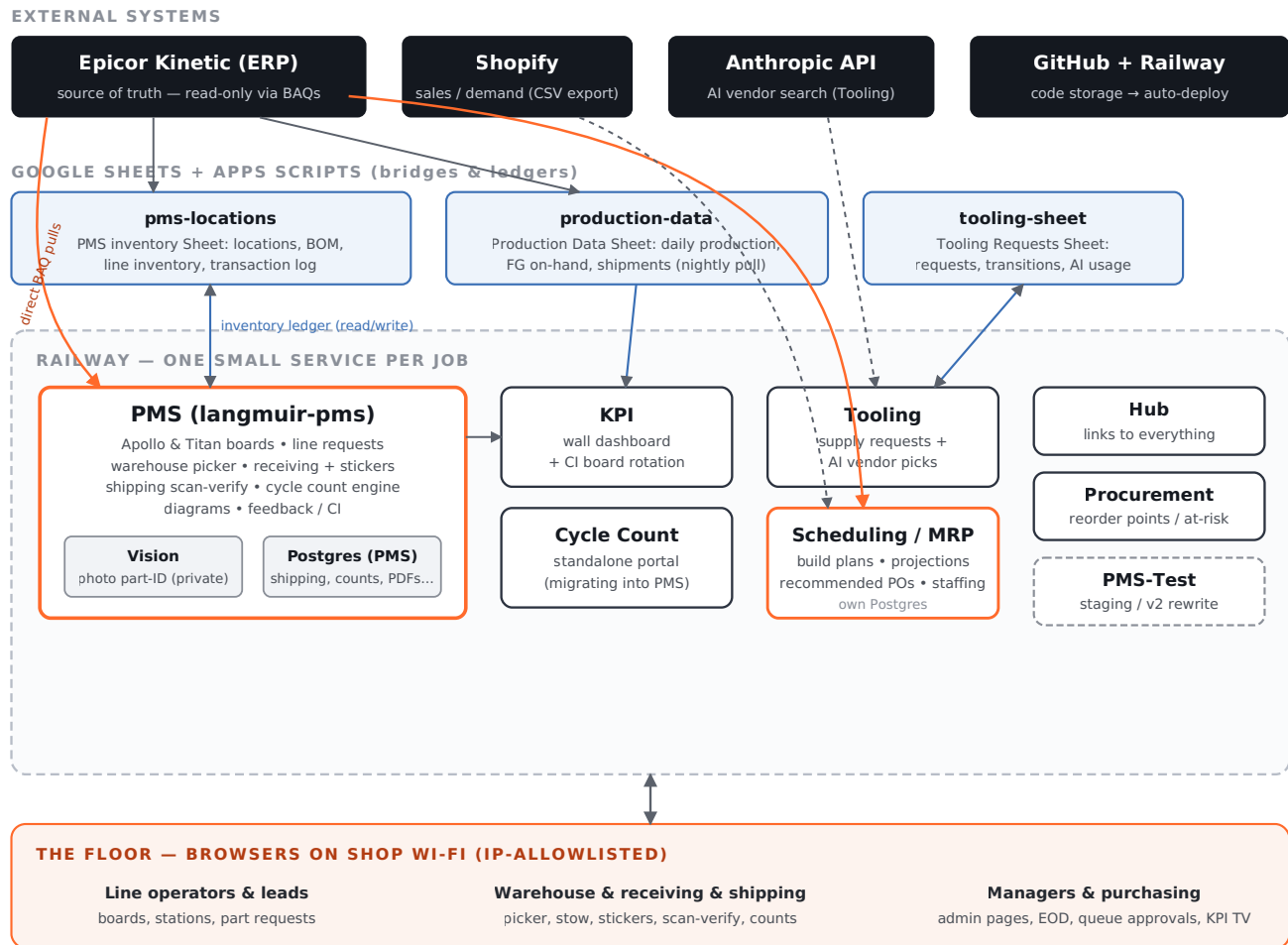


Figure 1 — The whole platform on one page. Orange borders mark the two most important services; blue boxes are Google-Sheet bridges; the dashed container is Railway.

Three observations to anchor your mental model. First, **PMS is the hub**: the KPI board reads shipping stats and the feedback board from it, the standalone Cycle Count app shares its inventory Sheet, and Vision shares its database and is only reachable through it. Second, **Epicor data flows one way — down**. Apps pull from Epicor on timers; nothing writes back automatically. When Epicor needs updating (say, after a cycle count), a human enters it, guided by an export the system prepares. Third, **each service deploys independently**: pushing a change to KPI cannot restart PMS, which means the floor screens stay up while you work elsewhere. That isolation is deliberate and worth protecting.

1.3 The design philosophy

Every app in the family follows the same house style, and understanding it will make every codebase feel familiar:

Small and plain beats clever. Each service is a single `server.js` plus plain HTML pages in a `public/` folder — no React, no build step, no framework. Pages poll the server or hold a WebSocket open. This is why Claude (and you) can read an entire application in one sitting.

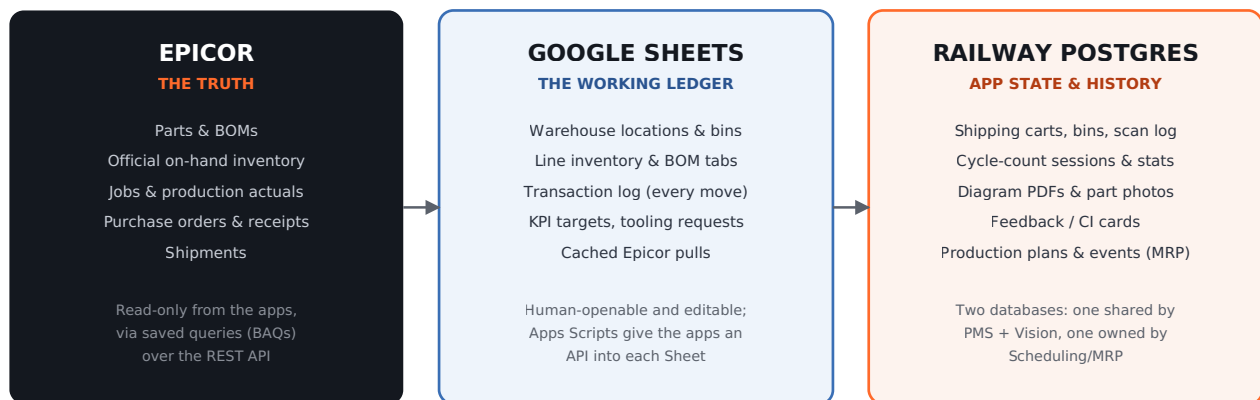
One service, one job, one repo. Each app has its own GitHub repository and its own Railway service, so deploys never collide. The trade-off is a little duplication (each app re-implements the same IP allowlist), accepted on purpose.

Low-ceremony security that can't lock out the floor. Access control is a shop-Wi-Fi IP allowlist (`ALLOWED_IPS`) that *fails open* if unset — a fresh deploy can never brick the kiosks. Sensitive actions use short PINs (picker login, manager admin, purchaser queue) rather than user accounts. Server-to-server calls use shared secret tokens.

Epicor is the source of truth; apps are advisory. The apps read Epicor through BAQs and recommend actions; they do not write back. Inventory corrections reach Epicor via a human keying in numbers from an export.

Built for turnover. Explainable logic beats optimal logic. Data feeds are configuration, not code (adding an Epicor query to a service is usually just an environment-variable change). Every significant module has a design document sitting next to the code (`DESIGN.md` , `SHIPPING_DESIGN.md` , `MODULE_PLAYBOOK.md` ...) — read those first when exploring.

1.4 The three data stores



Data generally flows left to right: Epicor truth → working copies → application records. Nothing flows automatically back into Epicor.

Figure 2 — Where data lives, and why. When numbers disagree, Epicor wins; the cycle-count process exists to pull the other two back in line with reality.

A common early confusion is why inventory seems to live in two places. The answer: the **Google Sheet tracks physical reality on the floor at bin level** (part X sits in location Y, line Z holds N of them) and is updated in real time by every stow, pick, and transfer. **Epicor tracks official company inventory** at a coarser level and moves on business transactions. They drift apart — parts get miscounted, scans get missed — and the cycle-count workflow (section 3.3) measures that drift daily,

corrects the Sheet on the spot, and hands the ops director a worklist to correct Epicor. The Postgres databases never claim to hold inventory truth at all; they record what the apps *did*.

Infrastructure & How Changes Go Live

2.1 Where everything lives

There are four places a piece of this system can live, and you have access to all of them:

The Cework workspace folder (`~/Documents/Claude/Projects/Langmuir Production Management System`) is the local copy of everything on the Mac. Each application sits in its own subfolder, each subfolder is its own git repository, and this is where you and Claude edit code. The folder also holds the workspace guide (`CLAUDE.md` — Claude reads this automatically every session), helper scripts, and working documents.

GitHub stores the official copy of each repository under the `BrendanLangmuir` account. The Mac authenticates with a dedicated work SSH key through a host alias called `github-langmuir`, so work repos and any personal repos never mix credentials. Every work remote looks like `git@github-langmuir:BrendanLangmuir/<repo>.git`.

Railway runs the live services. Each service watches its GitHub repository's `main` branch: when a commit lands there, Railway rebuilds and restarts that one service automatically, usually within a minute or two. Railway is also where environment variables (configuration and secrets) are set per service, and where the Postgres databases live.

Google Apps Script (`script.google.com`) hosts the three scripts bound to Google Sheets. Their code is mirrored into the workspace under `google-scripts/` and synced with a command-line tool called `clasp`.

2.2 The registry: config.sh

The file `scripts/config.sh` is the single source of truth for what belongs to this system. Every helper script reads it. If a repo or script isn't registered here, the tooling doesn't know about it.

Short name	Folder	GitHub repo	Railway service
pms	<code>pms/</code>	LangmuirPMS	langmuir-pms
pms-test	<code>pms-test/</code>	LangmuirPMS_Test	langmuir-pms-test
kpi	<code>kpi/</code>	LangmuirProductionKPI	langmuir-kpi
tooling	<code>tooling/</code>	langmuir-tooling	langmuir-tooling
cyclecount	<code>cyclecount/</code>	CycleCount	langmuir-cyclecount
scheduling	<code>scheduling/</code>	LangmuirScheduling	langmuir-scheduling
vision	<code>vision/</code>	LangmuirVision	langmuir-vision

The Hub (`langmuirhub`) and Procurement (`Langmuir-procurement`) apps live as clones inside the "Reorder points and stockout indicator" folder and are not yet registered in `config.sh` — worth adding when you touch them (section 3.7).

Script name	Bound to	Feeds
pms-locations	PMS inventory Sheet	PMS, Cycle Count, Scheduling
production-data	Production Data Sheet	KPI board, Scheduling (demand)
tooling-sheet	Tooling Requests Sheet	Tooling app

2.3 The life of a code change

This is the most important mechanical process in the whole handbook. Every change to a live application follows the same five steps:



Figure 3 — The life of a code change. Claude can edit but cannot deploy; you are the approval gate at steps 3-4.

Why can't Claude just push the change itself? Cowork's sandbox can read and write files in the workspace, but it cannot reliably run `git push` or `clasp push` against the mounted folder. This limitation is treated as a feature: it guarantees that a human reviews and executes every deployment. Claude's job ends at "here is the exact command to paste"; your job is to read the diff summary, ask questions until you understand it, and then run the command.

PRACTICAL HABIT

Before starting edits on an app you haven't touched in a while, pull the latest first: `./scripts/sync-repo.sh pms pull`. All Terminal commands run from the workspace folder: `cd ~/Documents/Claude/Projects/Langmuir\ Production\ Management\ System`. To see uncommitted changes across everything at once, run `./scripts/status.sh`.

2.4 Apps Script deployments (and the versioning gotcha)

The three Google Apps Scripts follow the same edit-review-push flow, but with `clasp` instead of git: `./scripts/sync-gscript.sh pms-locations push`. There is one crucial difference, and it is the single most common "why didn't my change take effect?" trap in this system:

GOTCHA: WEB-APP VERSIONS

An Apps Script deployed as a **web app** (which all three are — that's what gives them their `/exec` URLs) serves a *published version*, not the latest code. After `clasp push`, functions run from triggers and the editor use the new code immediately — but the `doGet` / `doPost` web endpoints keep serving the old version until you open the script at `script.google.com` → Deploy → **Manage deployments** → edit the deployment → select **New version** → Deploy. If an app that consumes the script (PMS, KPI, Cycle Count, Scheduling) isn't seeing your change, this is almost always why.

Secrets for the scripts (Epicor usernames, passwords, API keys) are never in the code. They live in each script's **Script Properties** (Project Settings → Script Properties in the Apps Script editor). If credentials rotate, that is where to update them.

2.5 Helper scripts reference

Script	What it does	Usage
<code>status.sh</code>	Shows pending git changes across every repo and clasp status for every script	<code>./scripts/status.sh</code>
<code>sync-repo.sh</code>	Pull, push (commit + push, message required), or status for one repo; on push it tells you which Railway service will restart	<code>./scripts/sync-repo.sh <name> <pull push status> ["msg"]</code>
<code>sync-gscript.sh</code>	clasp pull/push/status for one Apps Script	<code>./scripts/sync-gscript.sh <name> <pull push status></code>
<code>push-pms.sh</code>	Hardened one-command push just for PMS: fixes the remote, clears stale git locks, always pushes	<code>./scripts/push-pms.sh ["msg"]</code>
<code>deploy-scheduling.sh</code>	Orchestrated deploy of the Scheduling/MRP service: pushes the repo and the pms-locations script, then prints the manual follow-ups (publish script version, check env vars, verify health URLs)	<code>./scripts/deploy-scheduling.sh "msg"</code>
<code>set-scheduling-env.sh</code>	Sets all Epicor environment variables on the Scheduling Railway service in one go; verifies credentials against a live BAQ before touching Railway	<code>./scripts/set-scheduling-env.sh</code>
<code>bootstrap.sh</code>	One-time clone of every registered repo and script onto a fresh machine — this is how you'd rebuild the workspace on a new Mac	<code>./scripts/bootstrap.sh</code>

<code>first-push.sh</code>	One-time: turns a brand-new local folder into a GitHub repo (used when creating a new service)	<code>./scripts/first-push.sh <name> ["msg"]</code>
----------------------------	--	---

2.6 Access & security model

Understanding the layers keeps you from either over-worrying or under-worrying:

Network layer. Every user-facing service checks the caller's IP against `ALLOWED_IPS` (the shop's public IP). Off shop Wi-Fi, pages don't load. The check fails open when the variable is unset, so a misconfigured deploy inconveniences no one — but it also means you should confirm the variable survived if you ever recreate a service. Each service's `/healthz` endpoint is exempt and handily reports `yourIp`, which is how you discover the shop IP to allowlist.

Person layer. PINs gate sensitive actions: `PICKER_PIN` for warehouse login, `MANAGER_PIN` for the scheduling admin, `QUEUE_PIN` for tooling purchases, count-confirmation PINs for cycle count. Operators otherwise identify by typed name — deliberate low friction, with accountability via logs.

Service-to-service layer. Shared secret tokens: `VISION_TOKEN` (PMS → Vision), `FEEDBACK_TOKEN` (Cycle Count → PMS feedback). These live only in Railway environment variables.

Secrets rule. Never commit a password, API key, or token into a repository. Secrets go in Railway env vars or Apps Script Properties. Claude knows this rule; hold it to it.

The Applications

3.1 PMS — the hub of the floor

Folder `pms/` • repo `LangmuirPMS` • Railway service `langmuir-pms` • design docs: `MODULE_PLAYBOOK.md`, `SHIPPING_DESIGN.md`, `LINE_INVENTORY_DESIGN.md`, `BOM_SYNC_DESIGN.md`

PMS is the largest application and the one the floor lives in all day. It is one Express server (`server.js`) with four bolt-on modules — `shipping.js`, `cyclecount.js`, `diagrams.js`, `feedback.js` — sharing one Postgres database (`db.js`). The original core (boards, requests, picking) communicates over **WebSockets** so every screen updates instantly; the newer modules use simple polling. Start with who uses what:

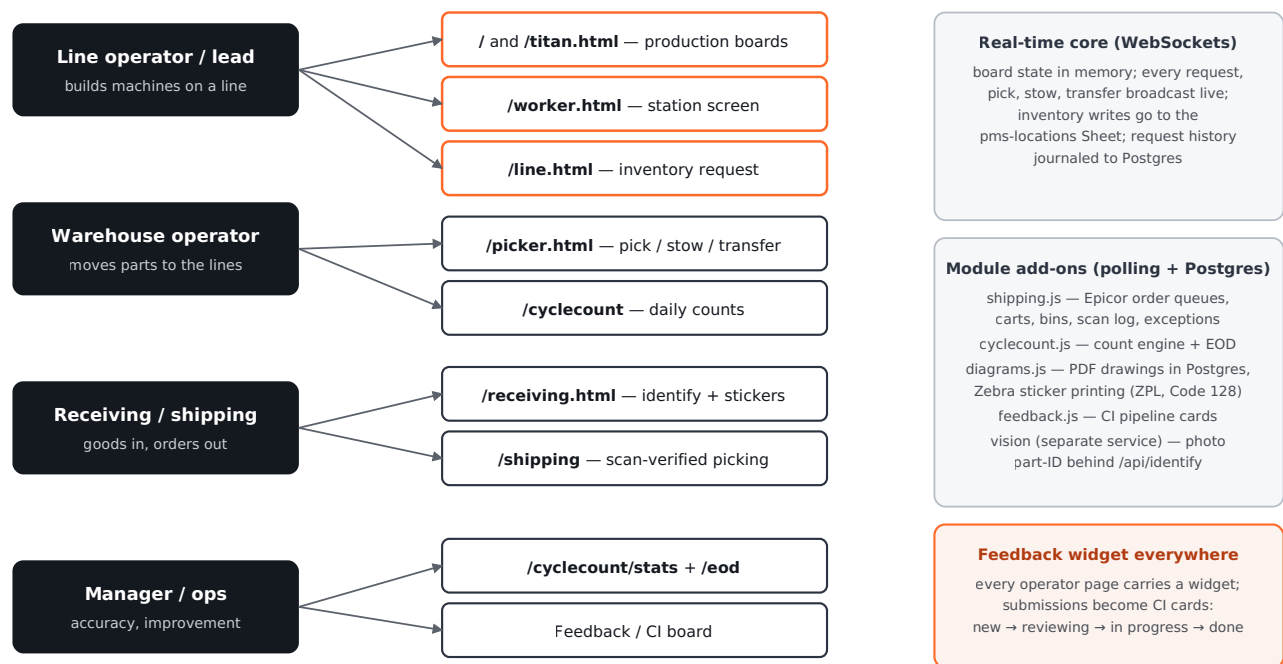


Figure 4 — PMS: who uses which page, and what sits behind them.

The production boards (Apollo & Titan)

The Apollo board (the app's home page) runs a station-based line on a 3-hour takt — a steady drumbeat that says a unit should complete every three hours. It shows units completed today, per-station status cards, and a live alerts rail. Line leads start, pause, and end the line; operators mark stations started or held (with a reason from a fixed list) from the station screen (`/worker.html`). The Titan board is the same idea adapted to a single-unit cycle at a 1.75-hour takt. Board state lives in server memory and every action travels over the WebSocket, which is why all screens showing a board agree within a second. The known lines across the system are Apollo, XF/PRO, TITAN, VULCAN, XR, MR1, and Shipping — these exact strings act as join keys across apps, so never rename a line casually.

Line inventory requests → warehouse picking

When a line runs short of a part, an operator submits a request on `/line.html` (part, quantity, priority). It appears instantly in the pick queue on `/picker.html`, the warehouse operator's main screen (PIN login). Fulfilling a pick does two things at once: it updates the Google Sheet ledger (subtract from warehouse location, add to the line) and journals the action durably. The picker page also handles **stow** (putting received stock into a location), **transfer** (moving stock between locations), and assigning "orphan" parts to lines. An auto-replenishment design also injects "Automated" requests when a line's tracked on-hand drops below its reorder point. A write-health panel on the picker page surfaces any Sheet writes that failed so they can be retried — watch it after Google hiccups.

Receiving — identify, verify, sticker

Receiving answers "what is this part, and how do we label it?" The operator looks the part up (or photographs it — see Vision, 3.5), checks the engineering drawing and reference photos, and prints **barcode stickers**: Code 128 barcodes (not QR) on 4"x2" labels via a Zebra ZD421 printer. The browser talks to the printer through the locally installed **Zebra Browser Print** agent — the server only generates the label code (ZPL); if stickers stop printing, the usual suspect is that agent on the receiving PC, not the server. Engineering drawings themselves are stored in the PMS Postgres database (uploaded PDFs matched to part numbers by filename convention), so they survive redeloys and stay inside the allowlisted network.

Shipping — scan-verified order picking

The shipping module exists so the wrong part physically cannot ship quietly. Order queues come from Epicor (a family of `ICM_ShippingDB` BAQs, refreshed every two minutes) split into Individual, Multipack, and Warranty active queues plus read-only counts for Software, Local, Hold, Deposit, and Freight. The workflow is bin-first: a picker **claims** orders into numbered bins on a cart, then picks; **every item must be scanned**, and the scan must match the expected part (by part-number barcode or a linked consumer UPC) or the screen flashes red and refuses. Unknown UPCs can be taught by scanning them against a known part once ("teach-by-scan"). A shipper marks bins shipped on the Bins tab; inventory is subtracted exactly once, at ship time. Scan mismatches and exceptions are logged — and exceptions automatically push that part onto the cycle-count priority list, which is a nice example of how the modules feed each other. The Activity tab shows the pick log and timing, and an aggregate stats endpoint feeds the KPI wall board.

Cycle count (the engine now lives here)

Covered fully in 3.3 — the short version: PMS hosts the current Postgres-backed cycle-count engine at `/cyclecount`, with the daily list generated ~6:00 AM, counts confirmed by PIN writing corrections to the Sheet, and an end-of-day worklist for updating Epicor.

Feedback / continuous improvement

Every operator page embeds a feedback widget. Submissions become cards in a pipeline (new → reviewing → in progress → done, or declined) with owner, due date, and votes; the board also renders on the KPI display's rotation. This is more important than it looks: it's the floor's channel into the improvement loop, and the planned automation work (Part 8) uses these cards as its work queue.

What can go wrong in PMS, in one paragraph

The Sheet is the inventory ledger, so Google outages or a broken pms-locations deployment degrade picking writes (watch the write-health panel). Board state is in memory, so a service restart mid-shift resets any line that was running — restarts are quick but time them kindly. The Zebra printing path depends on a local agent per PC. And because the allowlist fails open, an accidentally cleared `ALLOWED_IPS` silently exposes pages beyond the shop — worth a periodic glance at the Railway variables.

3.2 KPI — the wall board

Folder `kpi/` • repo `LangmuirProductionKPI` • Railway service `langmuir-kpi`

The KPI app is deliberately tiny — a static server of about a hundred lines. All the intelligence is in two pages. `KPI.html` is the finished-goods-by-line dashboard shown on the shop TV: it reads production, inventory, and shipment numbers from the **production-data Apps Script**, and shipping/inventory-accuracy stats directly from PMS (`/api/ship/kpi` , the one PMS endpoint open beyond the allowlist for this purpose). An edit modal writes target changes back to the Sheet. `ci.html` shows the continuous-improvement board, reading and voting through PMS's feedback API. A `ROTATE_SECONDS` setting flips the TV between the two pages.

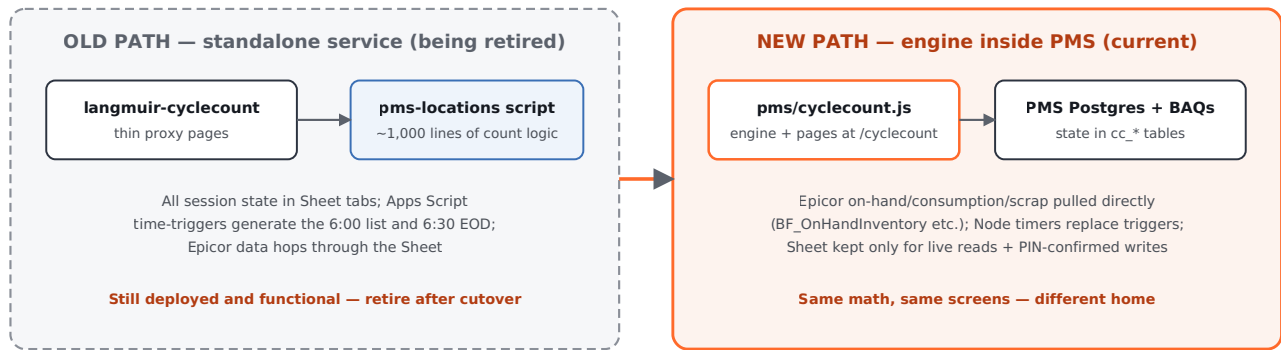
One implementation quirk worth knowing: browsers can't read server environment variables, so the server *injects* `PMS_URL` and `ROTATE_SECONDS` into the HTML as it serves it. If the shipping row on the TV goes blank, check that `PMS_URL` is set correctly on the Railway service.

3.3 Cycle Count — and its migration into PMS

Folder `cyclecount/` • repo `CycleCount` • Railway service `langmuir-cyclecount` • new engine: `pms/cyclecount.js`

Cycle counting is how recorded inventory stays honest. Each morning the system picks the parts whose official Epicor on-hand has drifted furthest from what the floor's tracking says, and a counter physically counts them. A confirmed count (PIN-gated) overwrites the warehouse and line records on the spot; at end of day, the parts still showing an Epicor discrepancy land on a worklist with a prepared adjustment so the ops director can key corrections into Epicor. Accuracy trends are on a stats page, and error rates feed the KPI board's inventory-accuracy tile.

The important thing to understand at handover is that **this feature exists twice**, because it is mid-migration:



Cutover plan: import history into Postgres (one-time endpoint), point users at PMS /cyclecount, delete the old Apps Script triggers, retire the standalone service.

Figure 5 — The cycle-count migration at handover. Confirm which URL the counters are actually using before changing either side.

HANDOVER CAUTION

Until the cutover is finished, a change to cycle-count behavior may need to consider both homes, and the two must not both run daily triggers against the same Sheet. When asking Claude for cycle-count changes, say which path you mean — it will also check the workspace memory, which tracks this migration.

3.4 Tooling — AI-assisted purchase requests

Folder `tooling/` • repo langmuir-tooling • Railway service `langmuir-tooling` • design doc: `tooling/DESIGN.md`

Tooling handles the things that don't have part numbers: gloves, drill bits, shop supplies. An operator fills a short mobile-friendly form (`/request`: who, which line, what, how many, why, optional photo). The server then asks **Claude via the Anthropic API** — with web search restricted to amazon.com, uline.com, and mcmaster.com — to return exactly three ranked purchase recommendations with links and prices. The purchaser opens the PIN-gated queue (`/queue`), reviews the options, and moves the request through a fixed lifecycle: *awaiting operator review* → *requested* → *approved* → *ordered* → *delivered* (or denied / re-research). Operators watch progress on `/status`. If the operator already knows the exact product, a "manual" path takes a URL and skips the full search.

Storage is a Google Sheet (via the tooling-sheet Apps Script) rather than a database — chosen deliberately so the purchaser can always open the raw requests in a familiar tool. Every AI call's cost is logged to an `ai_usage` tab, and a monthly budget gate (`AI_BUDGET_USD`, default \$50) quietly disables the AI path if spend crosses the line, with a spend badge on the queue page. So a "the AI stopped suggesting products" report is usually just the budget gate doing its job.

3.5 Vision — part identification by photo

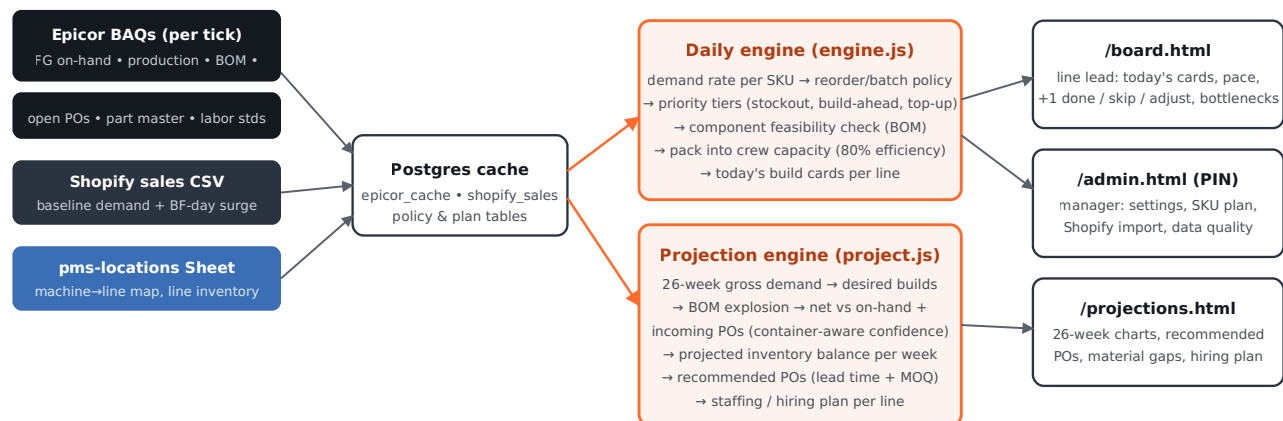
Folder `vision/` • repo LangmuirVision • Railway service `langmuir-vision` (no public URL)

Vision is the one headless service — no pages, no public domain, reachable only by PMS over Railway's private network with a shared token. When Receiving photographs an unknown part, PMS forwards the photo to Vision, which compares it against an index of every part photo ever taken at Receiving using a small image-recognition model (CLIP) that runs *inside* the service — photos never leave Langmuir's infrastructure. It returns the top visually similar candidates ranked by confidence; the receiver confirms the right one against the drawing and barcode. Each confirmation makes the index smarter. It shares PMS's Postgres database, reading the part-photo library and writing its own embedding and query-log tables. If it misbehaves, PMS degrades gracefully — Receiving just loses photo suggestions.

3.6 Scheduling / MRP — the planning brain

Folder `scheduling/` • repo `LangmuirScheduling` • Railway service `langmuir-scheduling` • docs: `MRP_ROADMAP.md`, `DESIGN.md`, `BAQ_BUILD_GUIDE.md`, plus `PROJECT_HANDOFF.md` at the workspace root

Where PMS executes today, Scheduling decides what tomorrow should look like. It is becoming a lean, in-house **MRP** (Material Requirements Planning) system: given what's selling, what's on hand, what's on order, and how much labor each unit takes, it computes what to build and buy, in what quantity, by when — and shows the answer instead of burying it. It is strictly *advisory*: it never writes back to Epicor.



A background "tick" refreshes all feeds and recomputes on a timer (default every 10 minutes); "Run engine now" on the admin page forces it.
Everything is advisory — the service reads Epicor and Shopify but never writes back to either.

Figure 6 — The Scheduling/MRP pipeline: feeds in on the left, two computation engines in the middle, three audiences on the right.

Two engines share one set of policy rules so they can never disagree. The **daily engine** answers "what should each line build today?" — it derives a demand rate per product (Shopify sales where mapped, Epicor shipments otherwise), computes reorder points and batch sizes *from the data* (nothing hand-entered), ranks candidate builds into priority tiers, checks component availability against the BOM, and packs the result into the day's actual labor capacity at an assumed 80% efficiency. Line leads see the result as prioritized cards with a pace bar; tapping +1 Done, Skip, or Adjust feeds actuals straight back into the plan. The **projection engine** answers "what does the next

26 weeks look like?" — exploding planned builds into component requirements, netting against on-hand and incoming purchase orders, and emitting projected inventory curves, recommended POs offset by each part's lead time, material-gap warnings, and a staffing plan. Its treatment of incoming POs is worth remembering: a PO only counts as *confident* supply once it's on a shipping container (a container ID in Epicor); an unshipped PO is optimistic at best. That distinction is the crux of Black Friday planning risk.

STATE AT HANDOVER

This is the newest and fastest-moving service. The code is ahead of some of its docs (README/DESIGN describe an earlier phase; trust `MRP_ROADMAP.md` and the code). Several Epicor feeds it wants (`BF_OpenJobs`, `BF_SalesOrders`, optionally `BF_Containers`) were still being built at handover — see Part 8. The "BAQ program" in Part 5 explains how each new feed gets wired in with configuration only.

3.7 Hub & Procurement

Clones live inside the workspace folder "Reorder points and stockout indicator" • Railway services `langmuir-hub`, `langmuir-procurement`

Hub is the simplest service in the family: a static landing page of cards linking to every internal tool. When you add or move an app, update the Hub so the floor can find it.

Procurement is decision support for purchasing: an at-risk/stockout dashboard, recomputed reorder-point recommendations, and AI-drafted supplier expediting messages. It grew out of a real problem — Asia lead times stretched by a week or more while Epicor's reorder points sat unchanged, causing stockouts. Its methodology (safety stock at a 99% service level from lead-time and demand variability) and the proposed refinements (first-receipt-only lead times; using container arrival dates to split supplier time from transit time) are documented in the two handoff notes in the same folder. Note its data still hops Epicor → Apps Script → Sheet → app, and a V3 redesign doc proposes going direct; it follows the same house conventions as everything else.

3.8 PMS-Test — the staging line

Folder `pms-test/` • repo `LangmuirPMS_Test` • Railway service `langmuir-pms-test` • design doc: `PMS_V2_DESIGN.md`

PMS-Test is not a copy of production — it is a **v2 rewrite of the PMS core loop** used as a safe staging ground. It deliberately omits the database-backed modules (no shipping, cycle count, diagrams, or feedback) and focuses on the four core jobs: getting material to lines on an SLA-based queue, inventory accuracy, production rhythm, and the Epicor loop. Its design encodes lessons learned from production data: every write is idempotent (a retried click can't double-subtract inventory), locations come from a registry instead of free text, open picks survive restarts by journaling to the Sheet, and cancellations require reason codes. Use it as the sandbox when you and Claude want to trial risky changes to core behavior before touching the real floor — and know that its migration to the single-app-shell design was only partial at handover.

The Google Apps Scripts

Apps Scripts are small programs that live inside Google Sheets. Here they play two roles at once: they give the web apps an API into each Sheet (a `doGet` URL to read, a `doPost` URL to write), and they run scheduled jobs that pull Epicor data into Sheet tabs. Their code is mirrored in the workspace under `google-scripts/`, so Claude can read and edit them like everything else — but remember the deployment-version gotcha from section 2.4.

pms-locations — the big one

Bound to the PMS inventory Sheet, this is by far the largest script (~3,100 lines) and the beating heart of physical inventory tracking. The floor manager can open the Sheet itself and see, in tabs, exactly what the apps see: `Locations` (what's in every bin), `BOM`, `Line Inventory`, `Transaction Log` (every move ever made), the replenishment queue, and cycle-count logs.

Its **read API** (`doGet`) returns the whole inventory picture in one payload — locations, on-hand, BOM, line inventory, machine→line map, replenishment queue — consumed by PMS constantly and by Scheduling for its map and line stock. Its **write API** (`doPost`) is how PMS records reality: `stow`, `subtract`, `bumpLine`, `assignOrphan`, request journaling, cycle logging, and the PIN-confirmed cycle-count corrections. On timers, it refreshes Epicor on-hand hourly into a `BAQ_Data` tab and runs the nightly line-consumption job that decrements line inventory based on what Epicor says each line actually consumed (with scrap accounted for).

production-data — the KPI feed

Bound to the Production Data Sheet. Every night (~6 AM, lock-guarded) it pulls three Epicor BAQs into tabs: daily production, finished-goods on-hand, and shipments. Its `doGet` serves that data plus two human-maintained tabs — `Targets` (the per-line tiles, colors, and daily targets shown on the TV) and `Stockouts` — to the KPI board, which polls every 30 minutes. The Scheduling service can also read shipments from it as a demand fallback. If the TV shows stale numbers, check this script's sync log and trigger first.

tooling-sheet — the tooling backend

Bound to the Tooling Requests Sheet, with three tabs the script itself creates: `requests`, `transitions` (an append-only audit of every status change), and `ai_usage` (cost log). It enforces the request state machine server-side, so even a buggy app can't move a request through an illegal jump. Consumed only by the Tooling app.

CREDENTIALS REMINDER

pms-locations and production-data hold Epicor credentials in Script Properties (never in code). Each script has a one-time setup function that seeds them. If Epicor credentials rotate, update Script Properties in *both* scripts, plus the `EPICOR_*` Railway variables on PMS and Scheduling.

The Epicor Connection (BAQs)

A **BAQ** (Business Activity Query) is a saved query built inside Epicor — think of it as a stored question like "give me every open purchase order line with its due date and container." Once built, Epicor exposes it at a REST URL, and any of our services can pull the rows with the right credentials. BAQs are the *only* way this platform touches Epicor, and they are read-only by construction.

How a feed gets wired in



Adding a data stream is configuration, not code — that was the point of the direct-sync design.
The service caches every pull in Postgres, so a brief Epicor outage doesn't blank the screens.

Figure 7 — Wiring a new Epicor feed into the Scheduling service (the same pattern PMS uses for its shipping and cycle-count feeds).

The BAQ inventory at handover

BAQ	What it returns	Consumed by	Status
BF_DailyProduction	Units produced per line per day	production-data → KPI; Scheduling	Live
BF_FGOnHandInventory	Finished-goods on-hand	production-data → KPI; Scheduling	Live
BF_FGShipments	Finished-goods shipments (legacy demand signal)	production-data; Scheduling fallback	Live (being replaced by Shopify)
BF_OnHandInventory	Component on-hand	PMS cycle count; pms-locations hourly refresh	Live
BF_DailyLineConsumption	Parts actually issued to each line's jobs	PMS cycle count; pms-locations nightly decrement	Live
BF_DailyScrap	Scrap by part/line	Same consumers as above	Live

ICM_ShippingDB_* family	Open sales orders by queue (Individual, Multipack, Warranty, Software, Local, Hold, Deposit, Freight)	PMS shipping (2-min poll)	Live
BF_PartBOM	Parent→component with quantity-per	Scheduling (explosion); PMS BOM sync design	Live
BF_OpenPOs	Open PO lines + container ID (transit signal)	Scheduling projections; Procurement	Live
BF_PartMaster	Lead time, MOQ, make/buy per part	Scheduling; Procurement	Live
BF_LaborStandards	Standard labor minutes per part	Scheduling capacity packing	Live
BF_POReceipts / lead-time stats	Receipt history for lead-time math	Procurement	Live; first-receipt refinement specced
BF_OpenJobs	Open manufacturing jobs (make-side receipts)	Scheduling netting	To build
BF_SalesOrders	Firm sales orders (independent demand)	Scheduling	To build
BF_Containers	Container → actual ETA	Scheduling receipt timing; Procurement	Optional, investigate

TWO LESSONS THE HARD WAY

JSON keys differ from labels. The REST API returns field IDs like `P0Rel_DueDate`, not the friendly column names you see in Epicor or an export. Always validate a live sample before trusting a new feed. **Container ID is the truth about arrival.** An open PO has a real ETA only once it's on a container (container ID $\neq$ 0); an unshipped PO's due date is a hope, not a plan. The planning engines already encode this — preserve that behavior in anything new.

Full per-BAQ build recipes — which Epicor tables to join, which fields and filters — live in `scheduling/BAQ_BUILD_GUIDE.md`. The pattern that works: duplicate an existing, working BAQ so REST access and company scoping come along free, then modify. Claude can guide you through any of these builds step by step; that's exactly how the existing ones were made.

Working with Claude Cowork

Cowork is how this system gets maintained and improved without a developer on staff. You describe what you want; Claude — with the entire workspace in front of it — investigates, proposes, and edits; you review and deploy. This chapter is about doing that *well*: knowing what Claude can see, what it can and can't do, what your requests actually set in motion, and how to stay in control of a live production system while moving fast.

6.1 What the workspace is

When you open this project in Cowork, Claude has the whole workspace folder mounted: every app's source code, every design document, the helper scripts, and this handbook's source material. Three files shape its behavior automatically:

`CLAUDE.md` at the folder root is the standing instruction sheet. Claude reads it at the start of every session. It encodes the working rules described in this handbook — the edit-then-propose-commands workflow, the real-vs-showroom triage, the never-commit-secrets rule. If you find yourself repeating an instruction to Claude across sessions, the fix is to ask Claude to add it to `CLAUDE.md`.

Persistent memory is a set of notes Claude keeps between sessions (project history, decisions, gotchas — for example, it tracks the cycle-count migration status). It loads automatically. When you and Claude make a significant decision, ask it to record the decision in memory so the next session starts warm.

The design docs next to each app (`DESIGN.md`, `SHIPPING_DESIGN.md`, `MODULE_PLAYBOOK.md`, `MRP_ROADMAP.md` ...) are where intent lives. Claude reads them before touching a module, and so should you — `MODULE_PLAYBOOK.md` in particular is the recipe used every time a new module gets added to PMS.

6.2 The working loop

The loop is the five-step flow from Figure 3, and it bears repeating because it is your safety model: **you ask → Claude edits locally → you review the diff → you paste the push command in Terminal → Railway deploys**. Claude cannot deploy anything by itself. Two useful elaborations:

Not every session is a change. Some of your most valuable sessions will be pure questions: "walk me through what happens when a shipping scan fails," "which services would be affected if the pms-locations Sheet were renamed?", "explain this week's cycle-count stats." Claude answers from the actual code and data files, which makes it the best tutor you have for this system. Use it that way liberally, especially in your first weeks.

Verify after deploying. After a push, give Railway a minute or two, then check the service's `/healthz` and click through the affected page. Claude will suggest the specific verification for each change — hold it to that habit, and tell it what you saw if something looks off. It can read the code to debug, but it cannot see the live site; your observations are its eyes.

6.3 Understanding impact before you approve

Every request you make lands somewhere in the system, and the blast radius differs enormously. This table is the impact map — the mental checklist to run before approving a change:

If the change touches...	What restarts / is affected	Risk level & what to watch
<code>pms/</code> anything	langmuir-pms restarts; in-memory board state resets (running Apollo/Titan lines reset mid-shift)	High. Deploy at shift edges or breaks. All floor screens depend on this one service.
<code>pms/</code> Postgres schema or Sheet-write code	Same, plus data-integrity risk	Highest. Ask Claude to explain the migration and rollback story before approving.
<code>kpi/</code> , <code>tooling/</code> , <code>cyclecount/</code> , <code>vision/</code>	Only that one service restarts	Low-moderate. Isolated by design; PMS unaffected.
<code>scheduling/</code>	langmuir-scheduling restarts; plans recompute (state is in Postgres, so nothing is lost)	Moderate. Verify <code>/healthz</code> and that the board repopulates after the next tick.
<code>google-scripts/pms-locations</code>	The inventory ledger's logic — PMS picking writes, cycle count, Scheduling's map all consume it	High. Multiple apps at once; needs clasp push <i>and</i> a new web-app version.
<code>google-scripts/production-data</code> or <code>tooling-sheet</code>	KPI board / Tooling app respectively	Low. Same version gotcha applies.
Railway environment variables	That service restarts with new config — no code change, no commit	Varies. Typos in <code>ALLOWED_IPS</code> , <code>EPICOR_*</code> , or URLs cause quiet degradation; double-check spelling.
A Google Sheet's tabs/ columns directly	No restart — but scripts and apps parse these tabs by name and position	Sneaky-high. Renaming a tab or column can break parsing with no deploy at all. Ask Claude what reads a tab before restructuring it.
<code>pms-test/</code>	Only the staging service	Minimal. This is the sandbox — experiment freely.

A GOOD REFLEX

Before approving anything non-trivial, ask Claude two questions: "*What else consumes what you just changed?*" and "*What would I check five minutes after deploying to know it worked?*" A good answer to both is your green light. Claude should volunteer this analysis; if it doesn't, ask.

6.4 How to ask well

Claude does its best work when a request carries three things: **the observed reality** ("pickers say the queue jumps around"), **the desired outcome** ("oldest requests should stay on top unless priority is higher"), and **the constraint** ("don't change how priorities are set on the line side"). Compare:

Weak request	Strong request
"Fix the picker page."	"On /picker.html, pickers report the pick queue reorders itself when new requests arrive, and they lose their place. Keep the sort stable: oldest first within each priority. Don't change how line leads set priority."
"Add a report."	"Managers want to see weekly shipping exceptions by part. Add a view to the shipping Activity tab — read-only, no new data collection. Show me the design before building."
"The KPI board is wrong."	"The KPI TV shows yesterday's production as 0 for TITAN, but the line ran. Trace where that number comes from and tell me whether it's the BAQ, the nightly sync, the Targets tab, or the board — before changing anything."

Three more habits that pay off. **Ask for a plan first on anything sizeable** — "propose an approach, don't edit yet" — so you can steer before code moves. **One concern per session thread** keeps diffs reviewable. And **when a change matters, ask for it on pms-test first**, then promote the same change to pms once you've clicked through it on staging.

6.5 Two worlds: real Langmuir vs. the showroom

Brendan also maintains **Finn Operations** — a separate consulting/demo brand whose showroom app is an *anonymized* clone of this system (generic line names, fake data) used for pitching capabilities to outside prospects. It lives in a **different folder** ("Anonymized ERP Clone") that is not part of this workspace, along with a private business-planning app and client pitch demos. This matters to you for exactly one reason:

THE ANONYMIZATION FIREWALL

Nothing real leaves this folder. Real customer names, real work-order data, real part numbers, Langmuir branding — none of it may be copied into anything showroom- or demo-related, ever, even as a "quick shortcut" to build a demo screen. The anonymization must always be deliberate. Claude's standing instructions enforce a triage on every request: real-ops requests proceed here; showroom requests require the other folder to be connected; ambiguous requests ("update the production board") get a clarifying question first. If you take over any demo work, keep that discipline — the cost of a leak is a customer's data in a stranger's pitch meeting.

6.6 Hard rules

Everything else in this chapter is technique; these five are law. **(1)** Never commit secrets — keys and passwords live in Railway variables or Script Properties only. **(2)** Never copy real data or branding into demo work (above). **(3)** Nothing deploys without a human reading the diff and running the command

— don't build automation that bypasses this gate; the in-flight MCP plan (Part 8) deliberately preserves it. **(4)** Epicor stays read-only from these systems; inventory corrections reach it through a human keying the prepared worklist. **(5)** PMS and KPI (and every other pair) stay separate repos and services — resist any convenience refactor that couples deploys.

Operational Playbook

7.1 What runs automatically

Much of the system is self-driving. Knowing the schedule tells you where to look when a number seems stale:

When (America/Chicago)	What happens	Where it runs
~6:00 AM nightly	production-data pulls daily production, FG on-hand, and shipments from Epicor into the KPI Sheet	Apps Script trigger
~6:00 AM	Cycle-count daily list generated + accuracy snapshot	PMS Node timer (new path)
~6:30 AM	Cycle-count end-of-day finalized → Epicor adjustment worklist ready	PMS Node timer
Nightly	Line-inventory decrement from actual Epicor consumption + scrap	pms-locations Apps Script
Hourly	Epicor on-hand refresh into the inventory Sheet's BAQ_Data tab	pms-locations Apps Script
Every 2 minutes	PMS shipping refreshes Epicor order queues	PMS
Every ~10 minutes (tick)	Scheduling refreshes all feeds and recomputes plans & projections	Scheduling
Every 90 s / 10 min	Vision embeds new part photos / refreshes its index	Vision
Every 15–30 s	Floor pages poll their servers (shipping queues, KPI board, etc.)	Browsers

7.2 Health checks & diagnostics

Every service answers `GET /healthz` from anywhere (it's exempt from the allowlist) and reports its own vitals plus `yourIp`. Your five-minute weekly sweep: open each service's `/healthz`, confirm a healthy response, and glance at the Railway dashboard for any service showing crash-loops or unusual memory. Deeper diagnostics, in the order you'll actually use them:

Scheduling looks empty or wrong → `/api/epicor-status` shows each BAQ's last sync and row count; a zero row count means the feed, not the engine. `/healthz` shows whether the machine→line map loaded (`skusFromMap` > 0) and Epicor is configured. The admin page's "Run engine now" forces a recompute after you fix a feed.

Inventory writes failing → the write-health panel on the picker page lists failed Sheet writes with a retry button. Persistent failures usually mean the pms-locations web app is erroring — check its executions log at script.google.com.

KPI TV stale → check the production-data script's trigger ran (executions log), then the Targets tab, then `PMS_URL` on the KPI service if only the shipping row is blank.

Vision quality → its `/stats` endpoint (token required) reports hit rates, index size, and last error.

7.3 Common failure modes and fixes

Symptom	Most likely cause	Fix
"My change didn't take effect" (Apps Script)	clasp push done, but no new web-app version published	script.google.com → Deploy → Manage deployments → New version (\$2.4)
Page won't load off-site	Working as intended — IP allowlist	Use shop Wi-Fi; to allow a new location, add its IP to <code>ALLOWED_IPS</code> (find it via /healthz <code>yourIp</code>)
Scheduling board plans nothing	Env vars missing after a redeploy (<code>EPICOR_*</code> , <code>EPICOR_BAQS</code> , <code>LOCATIONS_URL</code> , <code>KPI_URL</code>)	Run <code>./scripts/set-scheduling-env.sh</code> ; verify /api/epicor-status
Stickers won't print	Zebra Browser Print agent on that PC (server only generates ZPL)	Restart the agent / confirm printer connected; try another PC to isolate
Shipping scans all rejecting	Epicor queue cache stale or UPC not linked	Check /api/ship/status; teach unknown UPC by scanning it against the part
Apollo/Titan board reset mid-shift	PMS restarted (a deploy or Railway restart)	Boards restart quickly; deploy PMS at shift edges to avoid this
Cycle-count list didn't generate	Timer marker or BAQ cache issue (new path) / old Apps Script trigger still on (old path)	Check which path is live (§3.3); PMS logs in Railway; regenerate via the API
Tooling stopped giving AI suggestions	Monthly AI budget gate tripped	Raise <code>AI_BUDGET_USD</code> or wait for the month to roll; spend badge on /queue
Git push fails with lock error (PMS)	Stale <code>.git/index.lock</code>	<code>./scripts/push-pms.sh</code> handles it automatically
Numbers disagree between Sheet, app, Epicor	Normal drift — that's why cycle count exists	Add the part to today's count list; investigate only if drift recurs on the same part

7.4 Your first two weeks

Days 1-2 — read and click. Read Parts 1, 2, and 6 of this handbook. Open every live page next to its section in Part 3. Open the inventory Sheet and watch the Transaction Log while someone picks a part.

Days 3-4 — shadow the floor. Watch a picker fulfill requests, receiving print stickers, a shipping picker scan a bin, and a counter do the morning cycle count. The apps make far more sense after you've seen the physical motions they mirror.

Day 5 — first Cowork session, questions only. Ask Claude to walk you through the life of one part request end to end, then the life of one shipped order. Ask it anything you flagged while reading.

Week 2 — first supervised changes. Pick something cosmetic (a label, a color, a sort order) on a low-risk service (KPI or tooling). Run the full loop: request → diff review → push → verify. Then do a slightly bigger one on pms-test. Then run the weekly health sweep (§7.2) yourself. After that, you're operating.

Also in week 2 — confirm your access. You should be able to log into: Railway (all services + variables), GitHub (BrendanLangmuir org), Google (the three Sheets + script.google.com), Epicor (BAQ designer, read-only API key), the Anthropic console (tooling's API key), and the shop's Zebra printer PCs. Anything you can't reach, resolve while Brendan is still a phone call away.

In-Flight Projects at Handover

These are the initiatives that were live-but-unfinished in July 2026, in rough priority order. Each has its own documentation in the workspace; this is the orientation layer.

1. The BAQ program (Scheduling's missing feeds)

The MRP's remaining accuracy gains are blocked on Epicor queries, not code: `BF_OpenJobs` (make-side incoming supply), `BF_SalesOrders` (firm demand), and optionally `BF_Containers` (precise arrival dates). The specs are written (`scheduling/BAQ_BUILD_GUIDE.md`); the work is building them in Epicor's BAQ designer, validating sample rows, and adding names to `EPICOR_BAQS`. Claude can guide each build step by step. Also on this thread: retiring the legacy Sheets/KPI shipments path once direct feeds cover it.

2. Cycle-count cutover

Finish the migration in §3.3: confirm the floor uses PMS `/cyclecount`, run the one-time history import if not yet done, disable the old Apps Script triggers, then retire the standalone service. The workspace memory tracks exactly where this stands.

3. AI tech-support assistant

The biggest new build on the horizon, documented in `techsupport/FRAMEWORK.md` and `ACCESS_CHECKLIST.md` (distilled from TechSupport.pdf). The problem: ~3,100 support calls/month with 50% abandonment. The plan: keep Zendesk and RingCentral, build a Claude-powered layer on a central database (customers, machines/serials, warranty, tickets, calls, plus searchable docs) — AI triage and draft replies first, phone overflow later, humans always in control. Phase 0 is a data audit requiring no build; the access checklist is the to-do list of credentials and exports to gather. Two open decisions are flagged: where the database lives (Supabase vs. Railway Postgres) and the eventual phone approach.

4. MCP pipeline (Claude connected to live PMS)

`MCP_PIPELINE_PLAN.md` proposes letting Claude read live PMS data directly (feedback cards, pick history, shipping stats) through a token-protected endpoint, and eventually running a scheduled afternoon session that drafts fixes for approved feedback cards. The design explicitly preserves your role: the agent's write surface is staging and the feedback board only; production pushes stay human-gated. If you pursue this, phase 1 (read-only) is low risk and genuinely useful.

5. Reorder points & lead-time refinement

The procurement methodology work in the "Reorder points and stockout indicator" folder: implementing first-receipt-only lead-time measurement in the BAQ (specified in `BAQ_UPDATE_FirstReceipt.md`), investigating container-arrival data, and the Procurement app's V3 direction (direct Epicor data instead of the Sheet hop). Background, not urgent — but it protects against the stockouts that motivated it.

6. PMS v2 (pms-test)

The staging rewrite's ideas (idempotent writes, SLA queue, journaled requests) are partially staged; some already flowed into production. No deadline — treat pms-test primarily as your sandbox, and harvest its ideas into PMS when a related request comes up.

APPENDICES

A — Service & URL Reference

Service	Railway name	Key pages	Health
PMS	langmuir-pms	/ (Apollo), /titan.html, /worker.html, /line.html, /picker.html, /receiving.html, /shipping, /cyclecount (+ /count, /stats, /eod), feedback board	/healthz
KPI	langmuir-kpi	/ (KPI board), /ci	/healthz
Cycle Count (standalone)	langmuir-cyclecount	/, /count, /stats, /eod	/healthz
Tooling	langmuir-tooling	/request, /queue (PIN), /status	/healthz
Vision	langmuir-vision	none (headless; private network only)	/healthz
Scheduling / MRP	langmuir-scheduling	/board.html, /admin.html (PIN), /projections.html	/healthz, /api/epicor-status
Hub	langmuir-hub	/ (links to everything)	—
Procurement	langmuir-procurement	at-risk dashboard, recommendations	—
PMS-Test	langmuir-pms-test	mirrors PMS core pages (staging)	/healthz

B — Environment Variable Reference

Set per-service in Railway. This lists the ones you'll actually touch; each server.js documents the rest.

Variable	Service(s)	Purpose
ALLOWED_IPS	all user-facing	Shop IP allowlist (comma-separated); unset = open
DATABASE_URL	pms, vision (shared), scheduling (own)	Postgres connection
LOCATIONS_URL	pms, cyclecount, scheduling	pms-locations Apps Script /exec URL (inventory ledger API)
SHEETS_URL	tooling (also legacy on pms)	tooling-sheet Apps Script /exec URL
KPI_URL	scheduling	production-data /exec URL (shipments → demand fallback)
PMS_URL / PMS_FEEDBACK_URL	kpi / cyclecount	Where to reach PMS
EPICOR_HOST/COMPANY/USER/PASS/API_KEY	pms, scheduling	Epicor REST credentials
EPICOR_BAQS	scheduling	Comma-separated BAQ names to sync each tick
PICKER_PIN / MANAGER_PIN / QUEUE_PIN	pms / scheduling / tooling	Role PINs
FEEDBACK_TOKEN	pms + cyclecount	Server-to-server feedback posting
VISION_URL / VISION_TOKEN	pms + vision	Private photo-ID link
ANTHROPIC_API_KEY / AI_BUDGET_USD	tooling	AI vendor search + monthly budget gate
SHIP_SYNC_MS / SHIP_FREIGHT_VIAS	pms	Shipping queue poll rate / freight ship-via guard
CC_ONHAND_BAQ / CC_CONSUMPTION_BAQ / CC_SCRAP_BAQ	pms	Cycle-count BAQ names
ROTATE_SECONDS	kpi	TV rotation between KPI and CI boards

C — Glossary

Term	Meaning here
BAQ	Business Activity Query — a saved, REST-accessible query inside Epicor; the only way our apps read ERP data
Epicor Kinetic	The company ERP; source of truth for parts, inventory, jobs, POs
Railway	Cloud platform hosting all services; auto-deploys from GitHub main; also hosts the Postgres databases
Apps Script / clasp	Google's scripting inside Sheets / the command-line tool that syncs its code with the workspace
doGet / doPost	An Apps Script's web read/write endpoints (the /exec URLs)
Takt	The drumbeat interval at which a line should complete a unit (Apollo 3 h, Titan 1.75 h)
Kanban / reorder point	Replenish-when-below-threshold logic; here thresholds are computed from demand data, not hand-set
MPS / MRP	Master Production Schedule ("what to build when") / Material Requirements Planning (adds "what to buy when" via BOM math)
BOM explosion / netting	Converting planned builds into component needs / subtracting on-hand and incoming supply to find true shortfalls
MOQ / lead time	Minimum order quantity / days from order to arrival — both drive recommended POs
Container signal	An open PO is trusted supply only once assigned a shipping container in Epicor
Stow / pick / transfer	Warehouse verbs: put stock into a bin / fulfill a line request / move between bins
Cycle count / EOD	Daily physical counting of drift-prone parts / the end-of-day worklist for correcting Epicor
ZPL / Code 128 / Browser Print	Zebra's label language / the barcode type on part stickers / the local agent that lets browsers print to Zebra printers
WebSocket	A persistent browser-server connection; why PMS boards update instantly on every screen
Allowlist / fail-open	IP-based access control that permits everyone if unconfigured — safe for kiosks, worth auditing
Idempotent	Safe to retry — doing it twice has the same effect as once (v2 principle for inventory writes)
SLA	Service-level agreement — here, the promised fulfillment time attached to a part request
MCP	Model Context Protocol — the standard for connecting Claude to live systems (see Part 8)

Cowork workspace / CLAUDE.md	The mounted project folder Claude works in / its standing instruction file, read every session
---	--

Langmuir Production Systems — Owner's Handbook • prepared July 2026 for system handover • source material: the live workspace (CLAUDE.md, PROJECT_HANDOFF.md, per-app design docs, and the code itself). When this document and the code disagree, the code is right — and Claude can tell you what changed.